

RAGGER: Rethinking RAG via Hyper-layering Hybrid Search with Asynchronous Vector-DB Search

Kenneth EZUKWOKE *, Mohamed ELHAWARY, Thibaud ISHACIAN, Eric CHAU

Datategy
Research and Development
Paris, France.

Abstract

While Retrieval-Augmented Generation (RAG) has demonstrated significant potential in enhancing Large Language Models through external knowledge integration, current implementations face substantial trade-offs between efficiency and performance, particularly when scaling to large documents with millions of tokens. We present Hyper-layered Asynchronous Hybrid RAG (HAH-RAG), a novel framework that implements parallel hybrid search with asynchronous vector database querying. Our approach executes sparse and dense retrievals across multiple specialized layers, each optimized for different granularities of information retrieval. Through asynchronous vector database calls, we achieve a significant latency reduction while maintaining comprehensive search capabilities. Empirical evaluation demonstrates that HAH-RAG achieves a 66.6% reduction in energy consumption and CO₂ emissions compared to traditional hybrid approaches, while maintaining superior performance in factuality and correctness, scoring 0.745 and 0.645, respectively. The method demonstrates particular effectiveness in response time, achieving an average latency of 4.47 seconds - an improvement of 64.5% over existing methods. These results establish HAH-RAG as a promising direction for frugal implementation of RAG without compromising the quality of the performance.

Keywords: Retrieval-Augmented Generation, Asynchronous Hybrid Search, Large Language Models.

1 Introduction

Retrieval-Augmented Generation (RAG) has emerged as one of the most significant advances in AI, offering up-to-date and reliable external knowledge to enhance various tasks (Lewis et al., 2020). In the era of AI-Generated Content (AIGC),

RAG’s powerful retrieval advantages enable existing generative AI models to produce higher-quality output by incorporating an additional knowledge base (Li et al., 2023; Sheynin et al., 2023; Wu et al., 2024). Large Language Models (LLMs) have demonstrated exceptional abilities in language understanding and generation (Achiam et al., 2023; Chowdhery et al., 2023), paving the way for their adoption in various decision-making industries such as the semiconductor (Ezukwoke et al., 2024). However, they still face inherent limitations, including hallucinations and outdated internal knowledge (Zhao et al., 2023, 2024). These challenges are particularly prominent in domain-specific fields like medicine and law, where recent studies have shown hallucination rates ranging from 69% to 88% in responses to specific legal queries (Dahl et al., 2024).

To address these limitations, RAG has been integrated with LLM to enhance their performance in various tasks (Borgeaud et al., 2022; Izacard and Grave, 2021a; Khandelwal et al., 2020; Shi et al., 2023). RAG-augmented LLMs have shown particular promise in tasks that demand current and reliable knowledge, such as Question Answering, AI4Science, and software engineering. For example, Lozano et al. (2023) introduced a scientific QA method based on dynamic scientific literature retrieval, while MolReGPT leverages RAG to enhance ChatGPT’s in-context learning ability for molecular discovery (Li et al., 2023). Integration of RAG with LLM has also been shown to be effective in reducing hallucinations in conversational tasks (Shuster et al., 2021; Xu et al., 2022). As illustrated in the research, RAG-augmented dialogue methods can provide more accurate answers by retrieving relevant knowledge from external databases when dealing with out-of-scope queries.

2 Background

Retrieval-Augmented Generation (RAG) has emerged as a key advancement in artificial in-

*Corresponding author: kennedyczar@gmail.com

telligence, offering a solution to enhance Large Language Models (LLMs) by incorporating up-to-date and reliable external knowledge (Lewis et al., 2020; Borgeaud et al., 2022). Although LLMs have demonstrated remarkable abilities in language understanding and generation, they face inherent limitations, including hallucinations, outdated knowledge, and non-transparent reasoning processes (Zhao et al., 2023, 2024).

The state-of-the-art RAG-LLMs have shown remarkable success across various domains. In molecular discovery, RetMol (Wang et al., 2023b) and MoleculeSTM (Liu et al., 2023) leverage RAG for improved molecular property prediction and generation. Recent works by Li et al. (2024); Yepes et al. (2024); Zhang et al. (2023) have also demonstrated the effectiveness of RAG-LLMs in financial sentiment analysis and document processing. In the software engineering domain, advanced methods such as Atlas (Nashid et al., 2023) and DocPrompting (Zhou et al., 2022) have shown significant improvements in code generation and program repair tasks.

RAG addresses these challenges by enabling LLMs to dynamically access and integrate knowledge from external sources during generation.

The table 1 provides a comprehensive overview of the key components and considerations in RAG methods. The evolution from Naive RAG to Modular RAG shows the increasing sophistication of these methods, while the various retrieval methods and integration strategies offer different approaches to optimize performance based on specific use cases and requirements.

2.1 Evolution of RAG Methods

The development of RAG can be categorized into three distinct paradigms, namely: Native RAG, Advanced RAG, and Modular RAG.

2.1.1 Naive RAG

The initial RAG paradigm emerged as a straightforward "Retrieve-Read" framework (Ram et al., 2023), implementing a three-stage process: indexing, retrieval, and generation. During indexing, documents are processed into a uniform format, segmented into manageable chunks, and converted to vector representations for efficient similarity searches (Gao et al., 2023b). The retrieval phase employs the same encoding model to transform queries into vectors, computing similarity scores with indexed chunks. Finally, the generation stage

synthesizes the query and retrieved documents into a coherent prompt for the language model response (Lewis et al., 2020).

However, Naive RAG faces several limitations: retrieval precision and recall challenges often result in misaligned or irrelevant chunks, while the generation phase can suffer from hallucination issues where content is produced without proper support from the retrieved context (Asai et al., 2023). In addition, the integration of the information retrieved with different tasks can be challenging, sometimes resulting in disjointed or incoherent output.

2.1.2 Advanced RAG

Advanced RAG emerged to address the limitations of its predecessor by introducing sophisticated optimization strategies around pre-retrieval and post-retrieval processes (Izacard and Grave, 2021b). Key improvements include improved indexing techniques through sliding windows, fine-grained segmentation, and metadata incorporation (Gao et al., 2023b). Pre-retrieval optimization focuses on improving index structures and query transformation, while post-retrieval processes include reranking and context compression to improve the quality of retrieved information (Ma et al., 2023).

This paradigm introduces query optimization techniques such as query rewriting, transformation, and expansion (Wang et al., 2023a; Gao et al., 2023a). Post-retrieval enhancements include sophisticated reranking strategies and context compression methods to improve the relevance and usability of retrieved information (Hofstätter et al., 2023).

2.1.3 Modular RAG

The most advanced paradigm, modular RAG, offers enhanced adaptability and versatility by incorporating various strategies to improve its components (Asai et al., 2023). This approach introduces specialized modules for similarity searches, retriever fine-tuning, and various other functionalities (Gao et al., 2023b). Key innovations include: **Search Module**, which enables direct searches across various data sources using LLM-generated code and query languages. **Memory Module**, Leverages LLM’s memory to guide retrieval through iterative self-enhancement. **Task Adapter**, Tailors RAG to various downstream tasks through automated prompt retrieval and task-specific retriever creation (Cheng et al., 2023) and **Predict Module** reduces redundancy and noise by generating context di-

Table 1: Evolution of RAG methods, Retriever, Granularity and Integration with the pros and cons for each.

Category	Type/Paradigm	Key Features	Considerations
Evolution	Naive RAG	- Basic indexing - Simple retrieval - Direct generation	Pros: Simple implementation, low overhead Cons: Limited accuracy, prone to hallucinations Use-case: Simple QA, basic retrieval
	Advanced RAG	- Enhanced indexing - Pre/post retrieval processing - Optimized generation	Pros: Better accuracy, improved context handling Cons: Higher computational cost Use-case: Knowledge-intensive tasks, reasoning
	Modular RAG	- Specialized modules - Flexible architecture - Advanced integration patterns	Pros: High adaptability, task-specific optimization Cons: method complexity, resource intensive Use-case: Complex reasoning, multi-modal tasks
Retriever	Sparse Retrieval	- Word-based (TF-IDF, BM25) - No training required - Direct term matching	Pros: Simple, interpretable results Cons: Limited semantic understanding
	Dense Retrieval	- Embedding-based - Learned representations - Semantic matching	Pros: Better semantic understanding Cons: Training required, computationally intensive
Granularity	Document-level	- Full document retrieval - Comprehensive context	Pros: Complete context coverage Cons: Information overload
	Chunk-level	- Segmented retrieval - Balanced context size	Pros: Good efficiency balance Cons: Potential fragmentation
	Entity-level	- Knowledge graph based - Structured information	Pros: High precision Cons: Limited to structured data
Integration	Input-Layer	- Content concatenation - Direct augmentation	Pros: Easy implementation Cons: Input length limitations
	Output-Layer	- Post-processing - Result combination	Pros: Flexible implementation Cons: Limited reasoning depth
	Intermediate-Layer	- Deep integration - Advanced processing	Pros: Better reasoning Cons: Complex implementation

rectly through LLM (Yu et al., 2022).

2.2 Retrieval Methods

2.2.1 Retriever Types

Modern RAG methods employ three primary types of retrievers:

Sparse Retrieval : Implements word-based approaches using techniques such as TF-IDF and BM25 (Robertson et al., 2009). These methods rely on inverted index matching with raw data input, offering straightforward implementation but limited by their fixed-term-based nature. Although effective for direct text matching, they struggle with semantic understanding and context awareness (Gao et al., 2023b).

Dense Retrieval : Introduces neural networks to embed queries and documents in continuous vector spaces (Karpukhin et al., 2020). Key implementations include

- **Bi-encoder Architecture:** Implements two separate BERT-based encoders for queries and documents, as demonstrated in DPR (Karpukhin et al., 2020)
- **Single-encoder Models:** Pre-trained through contrastive learning on unaligned documents,

offering better generalization (e.g., Contriever) (Gautier et al., 2022)

- **Generation Model Integration:** Some methods use parts of the generation model as embedding layers to improve the alignment between the retrieval and generation processes (Lewis et al., 2020)

Hybrid Retrieval Hybrid retrieval methods combine the strengths of sparse and dense retrieval approaches to enhance the overall performance of RAG (Ma et al., 2023; Gao et al., 2023b). These methods leverage the complementary nature of lexical matching (sparse) and semantic understanding (dense) to improve retrieval accuracy and robustness. The sparse retriever implements traditional lexical matching using methods such as BM25 or TF-IDF (Robertson et al., 2009), while the dense retriever uses neural encoders to capture semantic relationships (Karpukhin et al., 2020).

Two main methods are used to combine the results of BM25 and dense retriever including **parallel** and **sequential retrievers**. Parallel processing is used to combine the results of both sparse and dense simultaneously in the input query, the results merged through a fusion mechanism (Khattab et al., 2022; Gao et al., 2023a). Sequential processing allows the output of one retriever to guide or filter

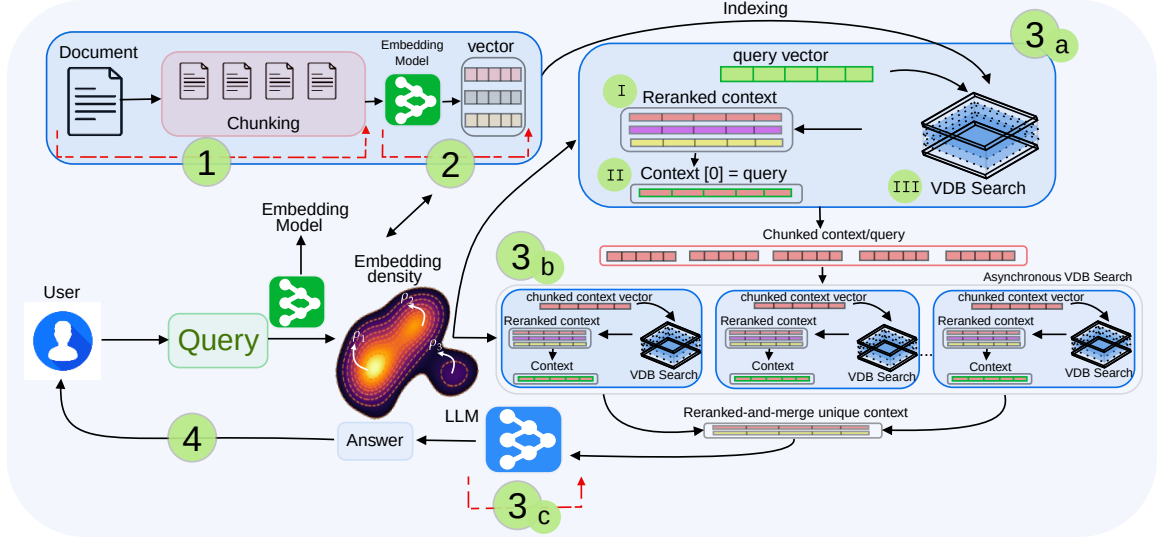


Figure 1: Proposed Hyper-layered Asynchronous Hybrid RAG (HAH-RAG).

the other’s search space (See 1). Through adaptive switching, the method dynamically chooses between sparse and dense retrieval based on the query characteristics (Ram et al., 2023). A Score-based Fusion method is used to combine the scores resulting from both methods as follows:

$$S_{final} = \alpha S_{sparse} + (1 - \alpha) S_{dense} \quad (1)$$

Where, α is a weight parameter that can be fixed based on empirical testing (trial-and-error), dynamically adjusted using query characteristics, or learned through supervised training. This fusion strategy gives the hybrid approach a significant edge in retrieval performance in both contextual understanding and robustness (Borgeaud et al., 2022; Lewis et al., 2020).

2.3 Architecture Innovations

Recent architectural advances in RAG-LLMs have focused on three primary integration approaches:

1. **Input-Layer Integration:** Methods like In-Context RALM (Ram et al., 2023) combine retrieved documents with the original input for joint processing. FID (Izacard and Grave, 2021b) improved on this by introducing independent document processing in the encoder, allowing scalable handling of multiple contexts.
2. **Output-Layer Integration:** methods like kNN-LM (Khandelwal et al., 2020) implement post hoc integration by interpolating LM

and retrieval-based distributions. REFEED (Yu et al., 2023) enhanced this approach by introducing an answer refinement mechanism using LLMs to evaluate retrieved information.

3. **Intermediate-Layer Integration:** More sophisticated approaches like RETRO (Borgeaud et al., 2022) introduced Chunked Cross Attention (CCA) layers for processing retrieved chunks within generator blocks, offering efficient handling of frequently retrieved information.

2.4 Training Strategies

State-of-the-art training approaches can be categorized into:

1. **Training-free Methods:** These approaches leverage retrieved knowledge during inference without additional training (Izacard and Grave, 2021b; Jiang et al., 2023; Khattab et al., 2022), making them computationally efficient and adaptable.
2. **Sequential Training:** Methods like RETRO (Borgeaud et al., 2022) and RALMs (Yoran et al., 2023) adopt a staged approach, first training one component (retriever or generator) before optimizing the other, allowing better component specialization.
3. **Joint Training:** Advanced approaches like RAG (Lewis et al., 2020) and REALM (Guu et al., 2020) optimize both retriever and generator simultaneously, enabling better synergy between components.

Our proposed RAG scheme leverages the training of the retriever at the time of creating the index without the need of repeating inference. A figure of the major comparison of our proposed scheme compared to existing methods is shown in Figure 2.

3 Hyper-layered Asynchronous Hybrid RAG (HAH-RAG)

Given the limitation that results from scaling traditional RAG schemes to huge documents and the loss of context that results from this, an improved RAG scheme is proposed. Hyper-layered asynchronous hybrid RAG (HAH-RAG) presents an innovative approach to retrieval-augmented generation by implementing a multi-layered, asynchronous hybrid search framework (see Fig. 1). This method improves traditional RAG methods through parallel processing and sophisticated document relevance scoring while keeping computational efficiency, precision trade-off at a *max - min*. Hence, the goal of HAH-RAG can be summarized into three main points: (I) Reducing the precision-speed trade-off of hybrid RAG for exponential data size, (II) Question contextualization for response improvement and factuality (III) Parallelizing the hybrid vector database search.

3.1 Notations

Let $\{d_i \in D\}_{i=0}^N$ represent a set of documents where N is the number of documents. We segment the document into smaller sets of documents such that $\{c_i \in C\}_{i=0}^N$ represents the fragments. Let $\{r_i \in R\}_{i=0}^K$ represent the retrievers running asynchronously, where K denotes the number of retrievers.

3.2 Technical Framework

3.2.1 Input Processing and Enhanced Context Processing

The scheme employs an advanced processing strategy that operates in multiple dimensions. Beyond basic document fragmentation, the system now incorporates query complexity analysis and dynamic parameter selection. The document processing follows:

$$C = \{c_1, c_2, \dots, c_n\} = f_{chunk}(D, \theta, k_{dynamic}) \quad (2)$$

where C represents the set of fragments, D is the input document, θ defines the fragmentation parameters, and $k_{dynamic}$ is dynamically selected based on the complexity analysis of the query. The segmentation function f_{chunk} incorporates semantic awareness through:

$$k_{dynamic} = \alpha \cdot complexity(query) + \beta \cdot len(D) \quad (3)$$

where α and β are tuning parameters adjusted based on empirical performance.

3.2.2 Caching Mechanism

The system implements a sophisticated two-level caching strategy:

$$C(q) = \begin{cases} cache[q] & \text{if } q \in cache \\ compute(q) & \text{if } relevance(q) > \theta_{cache} \\ retrieve(q) & \text{otherwise} \end{cases} \quad (4)$$

where θ_{cache} is the cache relevance threshold and the cache management follows:

$$relevance(q) = \gamma \cdot FREQ(q) + (1 - \gamma) \cdot REC(q) \quad (5)$$

Where $FREQ$ denotes the frequency and REC denotes the recency. The higher the relevance score the better the performance of the HAH-RAG scheme.

3.2.3 Retrieval

The multi-retriever ensemble scheme is defined with improved context filtering:

$$R = \{r_1, r_2, \dots, r_n\} \text{ where } r_i : C \rightarrow D_r \cdot Q(C) \quad (6)$$

Each retriever operates with parallel execution enhanced by context quality scoring:

$$P(c_i) = async\left\{\bigcup_{j=1}^n r_j(c_i)\right\} \cdot D_v(c_i) \quad (7)$$

3.2.4 Weighted Reciprocal Rank Fusion

The enhanced RRF algorithm now incorporates quality, Q and diversity, D_v :

$$RRF_{score}(d) = RRF_{bm25}(d) \cdot Q(d) + RRF_{faiss}(d) \cdot D_v(d) \quad (8)$$

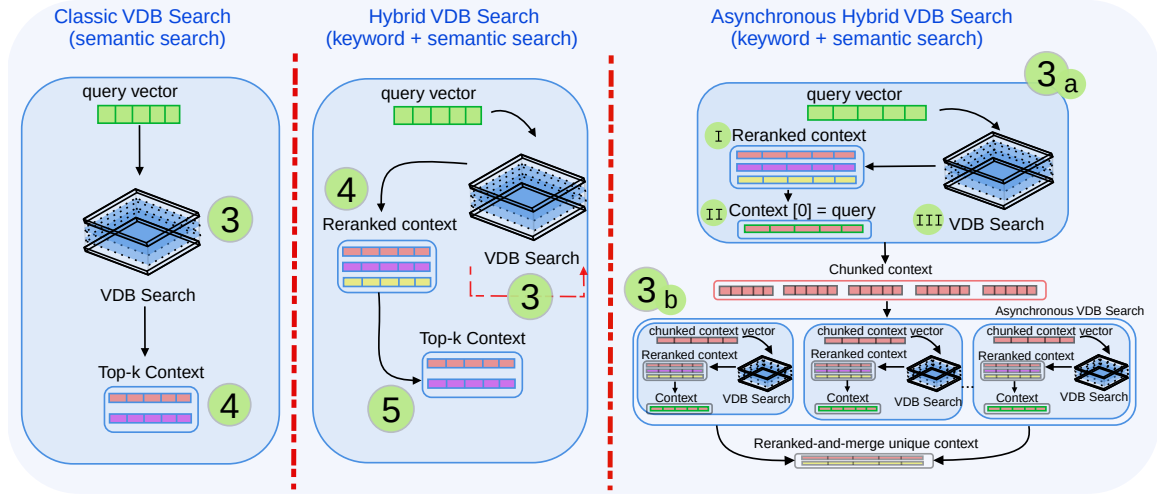


Figure 2: Comparing the evolution of the different RAG methods. HAH-RAG performs multiple asynchronous process in one step as opposed to the three steps hybrid method.

where:

$$RRF_{bm25}(d) = w_{bm25} \sum_{i=1}^k \frac{Q(d)}{\beta + r_{bm25_i}(d)} \quad (9)$$

$$RRF_{faiss}(d) = w_{faiss} \sum_{i=1}^k \frac{D_v(d)}{\beta + r_{faiss_i}(d)} \quad (10)$$

3.2.5 FlashReranker

The FlashReranker also takes into account context quality for computation, thus:

$$S_{final}(d) = \lambda \cdot RRF(d) + (1 - \lambda) \cdot F(d) \cdot Q(d) \quad (11)$$

where $Q(d)$ represents the quality score that incorporates relevance and diversity metrics.

3.3 Evaluation Metrics

For evaluating HAH-RAG’s performance across the selected datasets, we employ a comprehensive set of metrics that target different aspects of the proposed scheme against existing methods (see Fig. 2) (Järvelin and Kekäläinen, 2002; Barzilay and Lapata, 2008; Maynez et al., 2020; Gehrmann et al., 2022; Wang et al., 2018).

NDCG Computation The Normalized Discounted Cumulative Gain is computed as:

$$NDCG = \frac{DCG}{IDCG} \quad (12)$$

where:

$$DCG = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)} \quad (13)$$

$$IDCG = \sum_{i=1}^k \frac{2^{rel_i^*} - 1}{\log_2(i + 1)} \quad (14)$$

with rel_i representing relevance scores computed by cosine similarity between the retrieved and reference documents (Järvelin and Kekäläinen, 2002).

ROUGE Scoring The system implements comprehensive ROUGE metrics:

$$ROUGE-N = \frac{\sum_{gram_n \in ref} Count_{match}(gram_n)}{\sum_{gram_n \in ref} Count(gram_n)} \quad (15)$$

where N can be 1, 2, or L (longest common subsequence), providing multiple perspectives on the quality of generation (Lin and Hovy, 2003).

Latency System latency is measured as:

$$Latency = t_{end} - t_{start} + \sum_{i=1}^n CL_i \quad (16)$$

where CL_i , denoting cache lookup, represents the time spent checking the cache for existing results (following Eq. 3.2.2).

Fluency measures the grammatical correctness and natural flow of the generated text (Barzilay and Lapata, 2008). A perplexity-based fluency is adopted to compute the fluency, thus:

$$F = \exp \left(\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_{i-n+1}, \dots, w_{i-1}) \right) \quad (17)$$

where $P(w_i | context)$ is the probability of word w_i given its context, and N is the number of words.

Coherence evaluates the logical connection between sentences using cosine similarity of sentence embeddings, E (Barzilay and Lapata, 2008):

$$\text{Coherence, } c = \frac{1}{n-1} \sum_{i=1}^{n-1} \cos(E(s_i), E(s_{i+1})) \quad (18)$$

where $E(s_i)$ represents the embedding of sentence i . The responses close to 1 are very coherent, and those close to 0 lack coherence.

Relevance measures the semantic similarity between the question and generated response (Maynez et al., 2020):

$$\text{Relevance, } r = \cos(E(Q), E(A)) \quad (19)$$

where Q and A represent the question and answer respectively. Higher values indicate the correctness of the response with the question.

Factuality assesses the factual alignment between the generated text and source documents (Kryscinski et al., 2020):

$$\text{Factuality, } f = \max_i \cos(E(\text{generated}), E(\text{source}_i)) \quad (20)$$

Higher factuality score is preferred with scores ranging between $[0, 1]$.

HHEM (Hughes Hallucination Evaluation Model) combines similarity ($\bar{s}$) and factuality (f) scores (Gehrmann et al., 2022):

$$\text{HHEM} = \frac{\bar{s} \cdot f}{1 + \bar{s} \cdot f} \quad (21)$$

where $\bar{s}$ is the mean similarity score and f is the factuality score. This score is the reverse of the true hallucination due to ease of compute. When either similarity or factuality is low (closer to 0), it indicates potential hallucination, hence, HHEM score approaches 0.

Advanced HHEM incorporates coherence and relevance:

$$\text{Adv HHEM} = \frac{\bar{s} \cdot f \cdot c \cdot r}{1 + \bar{s} \cdot f} \quad (22)$$

where c is coherence score and r is relevance score. It rewards outputs that are not only factual but also coherent and relevant. Hence, higher values are preferred.

Correctness combines semantic similarity with key information overlap (Wang et al., 2018; Ribeiro et al., 2020):

$$\text{Correctness} = 0.5 \cdot ssc + 0.5 \cdot kis \quad (23)$$

where ssc is the semantic score and kis is the key info score and computed from the overlap of n-grams between generated and reference texts.

The metrics are scaled using a constant factor (scale metric) for standardization across different evaluation scenarios. Each metric produces scores in the range $[0, 1]$, where higher values generally indicate better performance, except for Advanced HHEM, where lower values suggest less hallucination.

3.4 Energy Consumption and Environmental Impact

The computation of energy consumption and environmental impact metrics is based on several key components (Henderson et al., 2020; Strubell et al., 2019):

3.4.1 Power Consumption

CPU Power (Patterson et al., 2022) is estimated based on CPU frequency and utilization:

$$P_{\text{CPU}} = P_{\text{base}} \cdot \frac{f_{\text{current}}}{f_{\text{max}}} \cdot \text{CPU}_{\text{util}} \quad (24)$$

where P_{base} is the base CPU power (typically 65W), f_{current} and f_{max} are current and maximum CPU frequencies, and CPU_{util} is the CPU utilization percentage.

GPU Power is computed either through direct measurement or estimation:

$$P_{\text{GPU}} = \begin{cases} P_{\text{measured}} & \text{if available} \\ 250W \cdot \text{GPU}_{\text{util}} & \text{otherwise} \end{cases} \quad (25)$$

where GPU_{util} is the GPU utilization percentage.

3.4.2 Energy Consumption

The total energy consumption is calculated by integrating power over time:

$$E_{\text{total}} = \sum_t \frac{(P_{\text{CPU}} + P_{\text{GPU}}) \cdot \Delta t}{1000 \cdot 3600} \text{ kWh} \quad (26)$$

where Δt is the sampling interval (typically 0.1 seconds).

3.4.3 CO₂ Emissions

CO₂ emission (Lacoste et al., 2019) is computed using country-specific carbon intensity factors:

$$\text{CO}_2 = \frac{E_{\text{total}} \cdot CI_{\text{country}}}{1000} \text{ kg} \quad (27)$$

where CI_{country} is the carbon intensity factor (in g CO₂/kWh) for the specific country. Since this experiment is conducted in France, we use a CI of 70 g CO₂ / kWh and a global average CI of 475 g CO₂/kWh.

3.4.4 Resource Metrics

The average resource utilization metrics are computed during the monitoring period.

$$\text{CPU}_{\text{avg}} = \frac{1}{N} \sum_{t=1}^N \text{CPU}_{\text{util}}(t) \quad (28)$$

$$\text{GPU}_{\text{avg}} = \frac{1}{N} \sum_{t=1}^N \text{GPU}_{\text{util}}(t) \quad (29)$$

$$\text{Memory}_{\text{avg}} = \frac{1}{N} \sum_{t=1}^N \text{Memory}_{\text{util}}(t) \quad (30)$$

where N is the total number of observations collected during the monitoring period.

These metrics are continuously monitored through a dedicated thread with a sampling interval of 0.1 seconds (sleep time), providing detailed resource usage profiles throughout the RAG execution time. The measurements account for both computational resources (CPU, GPU) and their environmental impact through energy consumption and CO₂ emissions calculations.

4 Experimentation

Setup. The machine used for this experimentation is an NVIDIA L40S comprising a GPU with 46GB usable RAM (12.4 CUDA version) that comes with a 60-core CPU.

4.1 Datasets

The evaluation of HAH-RAG follows a comprehensive scheme across multiple dimensions of performance using established benchmarks. To assess basic retrieval performance and knowledge integration, we employ three primary datasets: Natural Questions (NQ) (Kwiatkowski et al., 2019), which provides real user queries from Google Search, contains 307K training examples, 8K development

examples; TriviaQA (Joshi et al., 2017), offers a large-scale question answering dataset with evidence documents and comprises 95K questions with answers.

To evaluate the capability of HAH-RAG in handling complex reasoning and multistep retrieval, we use HotpotQA (Yang et al., 2018). HotpotQA is specifically selected to assess the effectiveness of our hyper-layered asynchronous scheme in managing multi-hop questions and made up of 113K Wikipedia-based question-and-answer pairs. For comprehensive response generation and context integration, we employ NarrativeQA (Kočíský et al., 2018), which is particularly relevant to evaluate how HAH-RAG handles chunk processing and maintains context coherence in longer responses.

4.2 Results

The average evaluation metric for all data sets is presented in Table 2 where HAH RAG is compared with the hybrid and naive RAG method. Higher values are preferred for all metrics except Latency and Advance HHEM, which measure hallucination. We also report mixed performance for where HAH RAG suffers to outperform both hybrid and naive methods. We hypothesize this may be as a result of the method used in computing coherence and the results itemized response of big LLMs.

4.3 Quantitative Analysis

HAH RAG demonstrates a remarkably fast response time, achieving an average latency of 4.47 seconds. This represents a substantial improvement of 64.5% compared to 12.60 s for hybrid RAG, and 63.5% compared to Naive RAG’s 12.25s. This significant reduction in latency does not come at the expense of accuracy, as evidenced by the evaluation metrics (see Table 2).

The factuality and correctness metrics particularly highlight HAH RAG’s strength in providing accurate and reliable responses. HAH RAG achieves a factuality score of 0.745, outperforming Hybrid RAG by 18.4% and Naive RAG by 34.5%. Similarly, its correctness score of 0.645 represents a substantial improvement of 38.1% over Hybrid RAG’s 0.467 and an even more significant 79.2% improvement over Naive RAG.

Table 3: Strongest correlations between evaluation metrics

Metric Pair	Corr. Coef.	Sig.
ROUGE-1 — ROUGE-L	0.998	***
ROUGE-1 — ROUGE-2	0.987	***
ROUGE-2 — ROUGE-L	0.985	***
Factuality — Correctness	0.871	***
HHEM — Correctness	0.712	***
HHEM — Factuality	0.654	***
Relevance — Factuality	0.587	**
Coherence — Fluency	0.543	**
NDCG — Correctness	0.521	**
Adv. HHEM — Coherence	0.489	**

Significance (Sig.): *** $p < 0.001$, ** $p < 0.01$

Table 2: Average Evaluation Metrics of different RAG method on NQ (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), HotpotQA (Yang et al., 2018) and NarrativeQA (Kočíský et al., 2018) datasets.

Metric	HAH RAG	Hybrid RAG	Naive RAG
Latency (s)	4.47 ± 2.28	12.60 ± 5.27	12.25 ± 3.73
NDCG	0.821 ± 0.178	0.821 ± 0.178	0.821 ± 0.178
ROUGE-1	0.246 ± 0.397	0.452 ± 0.521	0.291 ± 0.483
ROUGE-2	0.194 ± 0.389	0.431 ± 0.499	0.274 ± 0.483
ROUGE-L	0.246 ± 0.397	0.452 ± 0.521	0.291 ± 0.483
Fluency	0.895 ± 0.031	0.821 ± 0.067	0.923 ± 0.030
Coherence	0.198 ± 0.051	0.295 ± 0.027	0.321 ± 0.019
Relevance	0.715 ± 0.032	0.610 ± 0.073	0.729 ± 0.029
Factuality	0.745 ± 0.071	0.629 ± 0.030	0.554 ± 0.080
Correctness	0.645 ± 0.075	0.467 ± 0.024	0.360 ± 0.051
HHEM	0.354 ± 0.043	0.284 ± 0.019	0.236 ± 0.050
Adv. HHEM	0.047 ± 0.014	0.058 ± 0.009	0.055 ± 0.009

Best performing metrics shown in **bold**

Best performing metrics overall shown in **bold bleu**

The HHEM metric, as a measure of hallucination of the scheme, further supports HAH RAG’s effectiveness. HAH RAG achieves a high HHEM score of 0.354, surpassing hybrid RAG by 24.6% and naive RAG by 50%. This comprehensive metric suggests that HAH RAG better balances various aspects of performance, from accuracy to response quality.

Notice the strong pearson correlation (3) between factuality and correctness, and HHEM indicates that these metrics effectively capture the authenticity of the response returned by the pipeline (see Fig. 5 of the appendix section). There is a cluster of strong correlations between factuality, correctness, and HHEM metrics, with correlation coefficients ranging from 0.71 to 0.87. A complete table of strong correlation metrics is provided in Table 3.

4.3.1 Mixed Performance

While HAH RAG maintains competitive fluency performance with a score of 0.895 compared to 0.923 by Naive RAG and surpasses Hybrid RAG’s 0.821, it shows some limitations in coherence. The coherence score of HAH RAG of 0.198 ± 0.051 falls below hybrid and naive RAGs with respective scores of 0.295 and 0.321, indicating an area of potential improvement.

The ROUGE scores present interesting patterns across different datasets. Although the average ROUGE-1 score for HAH RAG is 0.246 and is lower than the hybrid RAG 0.452, the high standard deviations suggest significant variation across different types of queries. Notably, HAH RAG achieves perfect ROUGE scores on certain datasets, particularly in factual question answering scenarios, while showing lower performance in more complex narrative tasks.

4.4 Energy consumption and CO₂ emission

The results of energy consumption are particularly interesting, as the HAH RAG demonstrates a remarkable reduction 66% compared to the hybrid RAG (see Table 6). Empirical results show that HAH RAG achieves the lowest energy consumption at 0.0097 kWh, which is significantly lower than those of Hybrid RAG, 0.0290 kWh, and Naive RAG, 0.0271 kWh. This substantial reduction in energy consumption directly results in environmental benefits, as HAH RAG produces merely 0.00068 kg of CO₂ emissions, compared to 0.00203 kg and 0.00190 kg for Hybrid and Naive RAG, respectively. Low CO₂ emission is highly correlated with lower energy consumption and vice versa (see Fig. 4, Table 6). The error bars in Fig. 4 indicate the standard deviation.

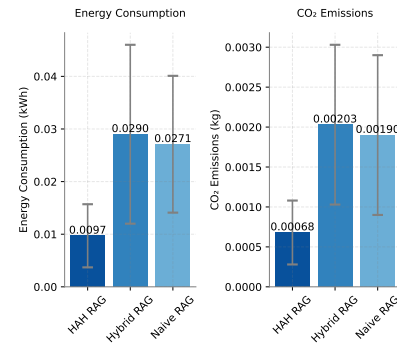


Figure 4: Comparing Energy and CO₂ emission.

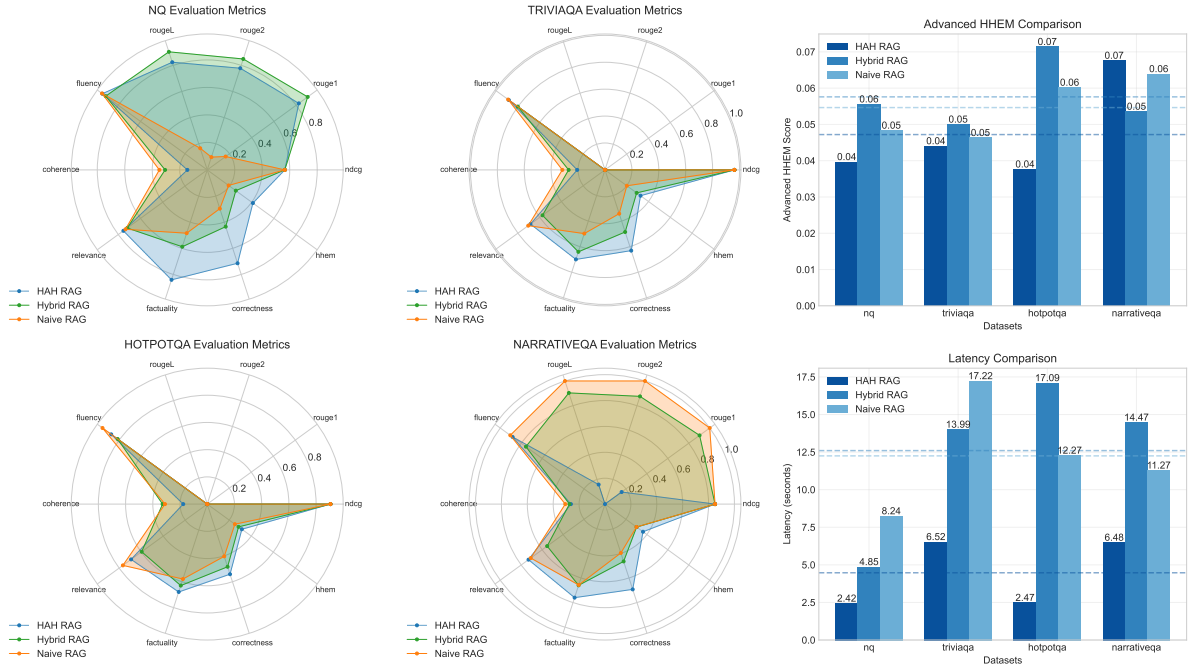


Figure 3: Evaluation metrics of different RAG method on NQ (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), HotpotQA (Yang et al., 2018) and NarrativeQA (Kočíský et al., 2018) datasets.

Table 4: Resource usage comparison of different RAG method on NQ (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), HotpotQA (Yang et al., 2018) and NarrativeQA (Kočíský et al., 2018) datasets.

Method	CPU Usage (%)	Memory Usage (%)	CPU Power (W)	Energy Total (kWh)	CO ₂ Emissions (kg)	Carbon Intensity*	GPU Util. (%)	GPU Mem. Used (MB)	GPU Mem. Max (MB)	GPU Mem. Total (MB)	GPU Power (W)
HAH RAG (Ours)	6.73	4.16	4.25	0.0097	0.00068	70	80.96	40408	40425	46068	202.83
	±0.07	±0.04	±0.04	±0.006	±0.0004		±4.12	±252			±9.51
Hybrid RAG	6.74	4.19	4.22	0.0290	0.00203	70	85.78	40389	40399	46068	214.75
	±0.01	±0.02	±0.05	±0.017	±0.001		±2.16	±239			±5.12
Naive RAG	6.76	4.19	4.29	0.0271	0.00190	70	86.31	40399	40409	46068	215.99
	±0.03	±0.01	±0.06	±0.013	±0.001		±0.91	±225			±2.44

* Carbon Intensity measured in g CO₂/kWh

Best performing metrics shown in **bold**

Table 5: Strongest Resource Metric Correlations

Metric Pair	Correlation
Energy Consumption — CO ₂ Emissions	1.000
GPU Utilization — GPU Power	0.998
CO ₂ Emissions — GPU Power	0.836
Energy Consumption — GPU Power	0.836
CO ₂ Emissions — GPU Utilization	0.827

Table 6: Environmental Impact Improvements

Percentage Improvements of HAH RAG	
Energy Consumption reduction — Hybrid	66.6%
Energy Consumption reduction — Naive	64.2%
CO ₂ Emissions reduction — Hybrid	66.5%
CO ₂ Emissions reduction — Naive	64.2%

HAH RAG performs optimally with a low CPU usage at 6.73% and a low memory usage at

4.16%. The GPU metrics further underscore HAH RAG’s efficiency, achieving the lowest use rate at 80.96% and the lowest GPU power consumption at 202.83W. This improved GPU efficiency suggests that the HAH RAG scheme better manages computational resources, potentially due to asynchronous calls.

Although Hybrid RAG shows marginal advantages in specific areas, such as CPU power consumption of 4.22W and GPU memory efficiency of 40389 MB, these improvements are relatively modest compared to HAH RAG’s substantial gains in overall energy efficiency and environmental impact. The result suggests that these minor advantages in specific metrics do not offset the significantly higher energy consumption and CO₂ emissions of the hybrid approach.

When examining the evaluation metrics across all datasets (Figure 3), HAH RAG maintains consistent performance while consuming fewer resources. Empirical results demonstrate that through careful scheme design and resource management, it is possible to achieve both improved efficiency and reduced environmental impact in large-scale information retrieval schemes.

5 Conclusion

We propose a novel RAG scheme, particularly in optimizing the balance between computational efficiency and retrieval accuracy. The empirical results reveal substantial improvements in multiple critical metrics, with HAH-RAG achieving a 66.6% reduction in energy consumption and corresponding CO₂ emissions compared to traditional hybrid approaches. This environmental benefit is achieved while maintaining superior performance in key metrics, including factuality (0.745 ± 0.071) and correctness (0.645 ± 0.075), demonstrating a frugal and efficient RAG framework without compromising retrieval quality.

The performance of the proposed scheme in latency reduction is particularly notable, with an average response time of 4.47 seconds representing a 64.5% improvement over existing methods. This efficiency gain is attributed to the innovative hyper-layering of the hybrid search scheme and optimized resource management, as evidenced by lower GPU usage at 80.96% and power consumption of 202.83W. Although there are areas for potential improvement, particularly in coherence metrics, the overall performance profile establishes HAH-RAG as a significant step forward in the design of environmentally conscious and frugal AI methods.

5.1 Perspectives

Investigation of fine-tuned optimization techniques (either embedding or model) could address current limitations in textual fluidity without compromising the frugal benefits.

Further research might explore the application of HAH-RAG's framework to other domains that require efficient large-scale information retrieval, such as legal document processing, medical document analysis, and scientific literature analysis. Additionally, the development of automated parameter optimization techniques could enhance the system's adaptability across different operational

contexts while maintaining its energy-efficient characteristics.

Acknowledgment

The corresponding author is the main contributor to this article and other authors contributed to the review and validation of these experiments. We acknowledge the contributions of Yassine YOUSFI (Engineering student, École des Mines de Saint-Étienne) and Enzo Damien (Data Scientist at Datatagy) in validating codes and experimentation, respectively.

References

- Josh Achiam, Steven Adler, Sandhini Agrawal, Lama Ahmad, Ilge Akkaya, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. [Self-rag: Learning to retrieve, generate, and critique through self-reflection](#). *arXiv preprint arXiv:2310.11511*.
- Regina Barzilay and Mirella Lapata. 2008. Modeling local coherence: An entity-based approach. *Computational Linguistics*, 34(1):1–34.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. 2022. Improving language models by retrieving from trillions of tokens. *International conference on machine learning*, pages 2206–2240.
- Daixuan Cheng, Shaohan Huang, Junyu Bi, Yuefeng Zhan, Jianfeng Liu, Yujing Wang, Hao Sun, Furu Wei, Weiwei Deng, and Qi Zhang. 2023. Uprise: Universal prompt retrieval for improving zero-shot evaluation. *arXiv preprint arXiv:2303.08518*.
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24:1–113.
- Matthew Dahl, Varun Magesh, Mirac Suzgun, and Daniel E Ho. 2024. Large legal fictions: Profiling legal hallucinations in large language models. *arXiv preprint arXiv:2401.01301*.
- Kenneth Ezukwoke, Anis Hoayek, Mireille Batton-Hubert, Xavier Boucher, Pascal Gounet, and Jérôme Adrian. 2024. [Big gcvae: decision-making with adaptive transformer model for failure root cause analysis in semiconductor industry](#). *Journal of Intelligent Manufacturing*.

- Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023a. Precise zero-shot dense retrieval without relevance labels. *ACL*, pages 1762–1777.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023b. [Retrieval-augmented generation for large language models: A survey](#). *arXiv preprint arXiv:2312.10997*.
- Izacard Gautier, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2022. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research*.
- Sebastian Gehrmann, Elizabeth Clark, and Thibault Selam. 2022. Repairing the cracked foundation: A survey of obstacles in human-ai collaboration. *arXiv preprint arXiv:2202.13452*.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval augmented language model pre-training. *International conference on machine learning*, pages 3929–3938.
- Peter Henderson, Jieru Hu, Joshua Romoff, Emma Brunskill, Dan Jurafsky, and Joelle Pineau. 2020. Towards the systematic reporting of the energy and carbon footprints of machine learning. *Journal of Machine Learning Research*, 21(248):1–43.
- Sebastian Hofstätter, Jiecao Chen, Karthik Raman, and Hamed Zamani. 2023. Fid-light: Efficient and effective retrieval-augmented text generation. *SIGIR*, pages 1437–1447.
- Gautier Izacard and Edouard Grave. 2021a. Distilling knowledge from reader to retriever for question answering. *ICLR 2021-9th International Conference on Learning Representations*.
- Gautier Izacard and Edouard Grave. 2021b. Leveraging passage retrieval with generative models for open domain question answering. *EACL 2021-16th Conference of the European Chapter of the Association for Computational Linguistics*, pages 874–880.
- Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20(4):422–446.
- Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. [Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension](#). In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1601–1611.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. *EMNLP*, pages 6769–6781.
- Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. 2020. Generalization through memorization: Nearest neighbor language models. *International Conference on Learning Representations*.
- Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. 2022. Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive nlp. *arXiv preprint arXiv:2212.14024*.
- Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. 2018. [The narrativeqa reading comprehension challenge](#). *Transactions of the Association for Computational Linguistics*, 6:317–328.
- Wojciech Kryscinski, Bryan McCann, Caiming Xiong, and Richard Socher. 2020. Evaluating the factual consistency of abstractive text summarization. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 9332–9346.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. [Natural questions: a benchmark for question answering research](#). *Transactions of the Association for Computational Linguistics*, 7:453–466.
- Alexandre Lacoste, Alexandra Luccioni, Victor Schmidt, and Thomas Dandres. 2019. Quantifying the carbon emissions of machine learning. *arXiv preprint arXiv:1910.09700*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.
- Jiatong Li, Yunqing Liu, Wenqi Fan, Xiao-Yong Wei, Hui Liu, Jiliang Tang, and Qing Li. 2023. Empowering molecule discovery for molecule-caption translation with large language models: A chatgpt perspective. *arXiv preprint arXiv:2306.06615*.
- Xiang Li, Zhenyu Li, Chen Shi, Yong Xu, Qing Du, Minghui Tan, Jun Huang, and Wei Lin. 2024. Alphafin: Benchmarking financial analysis with retrieval-augmented stock-chain framework. *arXiv preprint arXiv:2403.12582*.

- Chin-Yew Lin and Eduard Hovy. 2003. Automatic evaluation of summaries using n-gram co-occurrence statistics. In *Proceedings of the 2003 Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics*, pages 71–78.
- Shengchao Liu, Weili Nie, Chengpeng Wang, Jiarui Lu, Zhuoran Qiao, Ling Liu, Jian Tang, Chaowei Xiao, and Animashree Anandkumar. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12):1447–1457.
- Alejandro Lozano, Scott L Fleming, Chia-Chun Chiang, and Nigam Shah. 2023. Clinfo. ai: An open-source retrieval-augmented large language model system for answering medical questions using scientific literature. In *PACIFIC SYMPOSIUM ON BIOCOMPUTING 2024*, pages 8–23. World Scientific.
- Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query rewriting for retrieval-augmented large language models. *arXiv preprint arXiv:2305.14283*.
- Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1906–1919.
- Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-based prompt selection for code-related few-shot learning. *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2450–2462.
- David Patterson, Joseph Gonzalez, Quoc Le Holzinger, Hari Awan, Krste Asanovic, Titouan Parcollet, and Jeff Dean. 2022. Carbon footprint of training large language models. *Communications of the ACM*, 65(6):64–67.
- Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics*, 11:1316–1331.
- Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of nlp models with checklist. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912.
- Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389.
- Shelly Sheynin, Oron Ashual, Adam Polyak, Uriel Singer, Oran Gafni, Eliya Nachmani, and Yaniv Taigman. 2023. knn-diffusion: Image generation via large-scale retrieval. *ICLR*.
- Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. 2023. Replug: Retrieval-augmented black-box language models. *arXiv preprint arXiv:2301.12652*.
- Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. 2021. Retrieval augmentation reduces hallucination in conversation. *EMNLP (Findings)*, pages 3784–3803.
- Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2019. Energy and policy considerations for deep learning in nlp. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. 2018. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 353–355.
- Liang Wang, Nan Yang, and Furu Wei. 2023a. Query2doc: Query expansion with large language models. *EMNLP*, pages 9414–9423.
- Zichao Wang, Weili Nie, Zhuoran Qiao, Chaowei Xiao, Richard G Baraniuk, and Anima Anandkumar. 2023b. Retrieval-based controllable molecule generation. *ICLR*.
- Junda Wu, Cheng-Chun Chang, Tong Yu, Zhankui He, Jianing Wang, Yupeng Hou, and Julian McAuley. 2024. Coral: Collaborative retrieval-augmented large language models improve long-tail recommendation. *arXiv preprint arXiv:2403.06447*.
- Jing Xu, Arthur Szlam, and Jason Weston. 2022. Beyond goldfish memory: Long-term open-domain conversation. *ACL (1)*, pages 5180–5197.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*.
- Antonio Jimeno Yepes, Yao You, Jan Milczek, Sebastian Laverde, and Leah Li. 2024. Financial report chunking for effective retrieval augmented generation. *arXiv preprint arXiv:2402.05131*.
- Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. 2023. Making retrieval-augmented language models robust to irrelevant context. *The Twelfth International Conference on Learning Representations*.
- Wenhao Yu, Dan Iter, Shuohang Wang, Yichong Xu, Mingxuan Ju, Soumya Sanyal, Chenguang Zhu, Michael Zeng, and Meng Jiang. 2022. Generate rather than retrieve: Large language models are strong context generators. *arXiv preprint arXiv:2209.10063*.

- Wenhao Yu, Zhihan Zhang, Zhenwen Liang, Meng Jiang, and Ashish Sabharwal. 2023. Improving language models via plug-and-play retrieval feedback. *arXiv preprint arXiv:2305.14002*.
- Boyuan Zhang, Hongyang Yang, Tianyu Zhou, Muhammad Ali Babar, and Xiao-Yang Liu. 2023. Enhancing financial sentiment analysis via retrieval augmented large language models. *Proceedings of the Fourth ACM International Conference on AI in Finance*, pages 349–356.
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*.
- Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, et al. 2024. Recommender systems in the era of large language models (llms). *IEEE Transactions on Knowledge and Data Engineering*.
- Shuyan Zhou, Uri Alon, Frank F Xu, Zhengbao Jiang, and Graham Neubig. 2022. Docprompting: Generating code by retrieving the docs. *The Eleventh International Conference on Learning Representations*.

A Evaluation Dataset Sources

1. Knowledge-intensive QA:

- Natural Questions (NQ):
<https://github.com/google-research-datasets/natural-questions>
- TriviaQA:
<http://nlp.cs.washington.edu/triviaqa/>

2. Multi-hop Reasoning:

- HotpotQA:
<https://hotpotqa.github.io/>

3. Long-form Generation:

- NarrativeQA: <https://github.com/deepmind/narrativeqa>

Most datasets are provided in JSON format, and some offer additional formats such as CSV or SQL databases. Although each data set includes specific splits for training, validation, and testing purposes; we only use the data set evaluation in the data set evaluation for performance comparison.

B Further Quantitative Evaluation

B.1 Complete table of Energy consumption with CO₂ emission

We summarize the metrics on energy consumption and CO₂ emission of the HAH RAG in comparison with other RAG methods as follows:

- On the NQ dataset, HAH RAG achieves the lowest energy consumption of 0.0026kWh and CO₂ emissions, 0.00018kg.
- For HotpotQA, HAH RAG maintains efficient resource usage with only 0.0054kWh energy consumption
- GPU usage is notably lower for HAH RAG on both NQ (76.63%) and HotpotQA (78.18%) datasets. This can be attributed to the
- However, since Hybrid RAG does not support operation parallelization, it does better at CPU power efficiency on the NQ dataset (4.15W) and NarrativeQA (4.21W) than HAH RAG.

B.2 Dataset-Specific Performance

HAH RAG dominates in data-specific problems where both simple and complex reasoning are required. In fact, dataset-specific analysis reveals HAH RAG’s particular strengths in handling natural questions (NQ) and factual queries. For the NQ dataset, HAH RAG achieves outstanding performance in multiple metrics:

- **Factuality:** scores 0.841 compared to the 0.588 and 0.484 of Hybrid and Naive RAG, respectively.
- **Correctness:** scores 0.714 compared to 0.434 by hybrid and 0.296 by naive RAG.
- **Response time:** it scored the lowest latency at 2.42s, which is 2X faster than Hybrid and 3X faster than Naive RAG.

The performance on TriviaQA and HotpotQA datasets demonstrates the consistency of HAH RAG across different question types, maintaining strong factuality and correctness scores while significantly outperforming other methods in response time. For instance, on HotpotQA, HAH RAG’s latency of 2.47s represents an 85.5% improvement over Hybrid RAG and 79.9% over Naive RAG’s 12.27s.

B.3 Evaluation metrics correlation

The correlation matrix between the evaluation metrics is shown in Fig. 5.

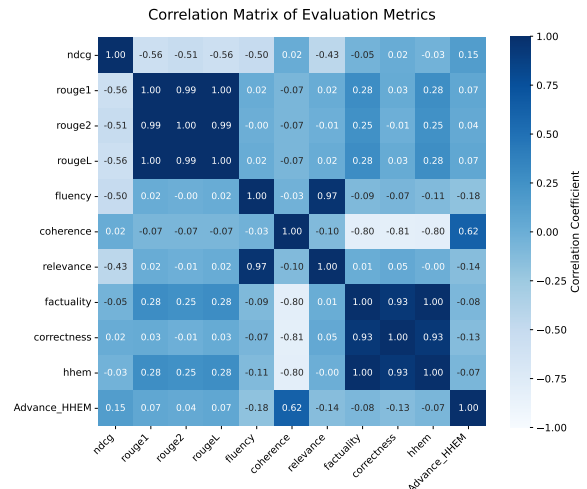


Figure 5: Correlation matrix between the evaluation metrics.

B.4 Resource metrics correlation

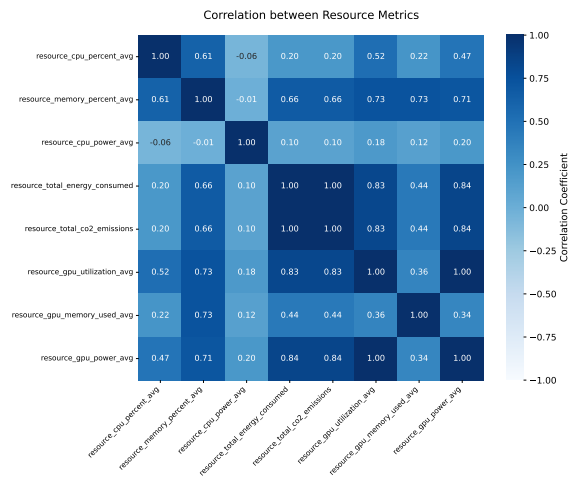


Figure 6: Correlation matrix between the resource metrics.

Table 7: Correlations with Total Energy Consumption

Metric	Correlation
CO ₂ Emissions	1.000
GPU Power	0.836
GPU Utilization	0.827
NDCG	0.670
Memory Usage	0.664
Relevance	-0.575
Fluency	-0.557
Coherence	0.538
Advanced HHEM	0.459
GPU Memory Used	0.439

Note: Values closer to ± 1 indicate stronger correlations.

Table 8: Comprehensive Performance Analysis of different RAG method on NQ (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), HotpotQA (Yang et al., 2018) and NarrativeQA (Kočíský et al., 2018) datasets.

Table 9: Resource Usage Comparison

Method	Dataset	CPU Usage (%)	Memory Usage (%)	CPU Power (W)	Total Energy (kWh)	CO ₂ Emissions (kg)	Carbon Intensity	GPU Util. (%)	GPU Mem. Used (MB)	GPU Mem. Max (MB)	GPU Mem. Total (MB)	GPU Power (W)
HAH RAG	NQ	6.62	4.10	4.30	0.0026	0.00018	70	76.63	40116.31	40126.00	46068.00	193.78
	TriviaQA	6.78	4.15	4.25	0.0154	0.00108	70	84.60	40306.05	40328.00	46068.00	211.05
	HotpotQA	6.78	4.18	4.19	0.0054	0.00038	70	78.18	40522.69	40560.00	46068.00	195.54
	NarrativeQA	6.73	4.19	4.28	0.0152	0.00107	70	84.45	40685.19	40686.00	46068.00	210.96
	Average	6.73	4.16	4.25	0.0097	0.00068	70	80.96	40407.56	40425.00	46068.00	202.83
Hybrid RAG	NQ	6.73	4.16	4.15	0.0055	0.00039	70	82.45	40092.95	40098.00	46068.00	206.84
	TriviaQA	6.77	4.19	4.25	0.0339	0.00237	70	86.76	40318.32	40324.00	46068.00	216.96
	HotpotQA	6.74	4.20	4.27	0.0415	0.00291	70	87.05	40483.10	40508.00	46068.00	217.81
	NarrativeQA	6.74	4.20	4.21	0.0350	0.00245	70	86.86	40661.83	40664.00	46068.00	217.39
	Average	6.74	4.19	4.22	0.0290	0.00203	70	85.78	40389.05	40398.50	46068.00	214.75
Naive RAG	NQ	6.80	4.17	4.30	0.0095	0.00067	70	85.00	40119.08	40140.00	46068.00	212.64
	TriviaQA	6.74	4.18	4.21	0.0419	0.00293	70	87.12	40325.68	40332.00	46068.00	217.92
	HotpotQA	6.75	4.20	4.35	0.0297	0.00208	70	86.75	40485.34	40488.00	46068.00	217.16
	NarrativeQA	6.76	4.20	4.30	0.0272	0.00191	70	86.38	40664.58	40676.00	46068.00	216.23
	Average	6.76	4.19	4.29	0.0271	0.00190	70	86.31	40398.67	40409.00	46068.00	215.99

Table 10: Evaluation Metrics Comparison

Method	Dataset	Latency	NDCG	ROUGE-1	ROUGE-2	ROUGE-L	Fluency	Coherence	Relevance	Factuality	Correctness	HHEM	Adv. HHEM
HAH RAG	NQ	2.42	0.564	0.824	0.778	0.824	0.943	0.145	0.754	0.841	0.714	0.411	0.040
	TriviaQA	6.52	0.962	0.000	0.000	0.000	0.884	0.206	0.685	0.700	0.631	0.326	0.044
	HotpotQA	2.47	0.906	0.000	0.000	0.000	0.872	0.176	0.692	0.679	0.542	0.314	0.038
	NarrativeQA	6.48	0.850	0.159	0.000	0.159	0.882	0.263	0.730	0.760	0.693	0.363	0.068
Hybrid RAG	NQ	4.85	0.564	0.903	0.848	0.903	0.913	0.307	0.719	0.588	0.434	0.257	0.056
	TriviaQA	13.99	0.962	0.000	0.000	0.000	0.801	0.270	0.574	0.640	0.485	0.291	0.050
	HotpotQA	17.09	0.906	0.000	0.000	0.000	0.813	0.328	0.596	0.629	0.484	0.285	0.071
	NarrativeQA	14.47	0.850	0.903	0.875	0.903	0.755	0.274	0.552	0.660	0.466	0.302	0.054
Naive RAG	NQ	8.24	0.564	0.166	0.098	0.166	0.946	0.348	0.735	0.484	0.296	0.192	0.048
	TriviaQA	17.22	0.962	0.000	0.000	0.000	0.890	0.317	0.705	0.498	0.342	0.201	0.046
	HotpotQA	12.27	0.906	0.000	0.000	0.000	0.951	0.310	0.766	0.579	0.404	0.251	0.060
	NarrativeQA	11.27	0.850	1.000	1.000	1.000	0.904	0.307	0.709	0.657	0.397	0.301	0.064